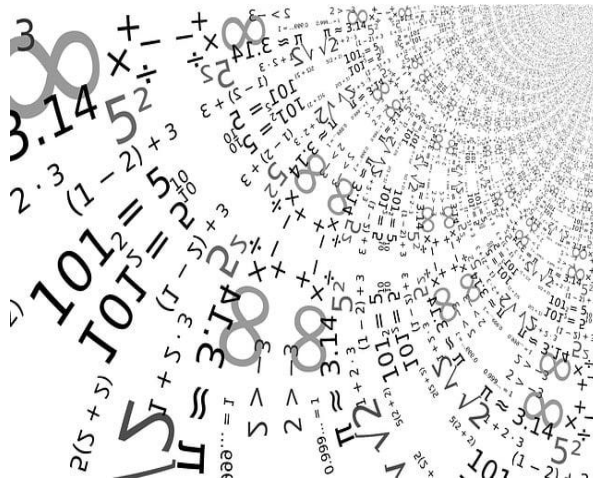


Kunskapskontroll 2

Sifferigenkänning: Maskininlärningsmodeller för multiklassifikationsproblemet, data-set MNIST



Katarina Persson Data scientist,

EC Utbildning

Kunskapskontroll 2

2025-03-23

Abstract

The project was carried out as part of the Machine Learning course at the Data Scientist training at the EC training in Gothenburg. The work included theoretical questions about machine learning and a detailed exploratory data analysis (EDA) of the MNIST dataset. At least two different models were developed, trained and evaluated to classify handwritten digits. An interactive application was designed with Streamlit, where the user can upload a file or draw a digit on a canvas to get a prediction. All work and the insights gained were carefully documented in a written report.

Innehållsförteckning

1	Inledning.....	1
2	Teori.....	2
2.1	Algoritmer vid klassificeringsproblem: One versus One, och One versus the Rest.....	2
2.2	Modeller inom multiklassificeringsproblem.....	2
2.3	Optimering och hyperparametrar.....	4
2.4	Ensemble-teori:	4
2.5	Utvärdering	4
3	Metod.....	5
3.1	EDA- MNIST.....	5
3.2	Modellval	7
3.3	Modell Utvärdering	8
3.4	Classification report.....	8
3.5	Confusion matrix	9
3.6	Prediktionstest baserat på 5 MNIST bilder.....	9
3.7	Optimering.....	10
4	Resultat och Diskussion	12
4.1	Streamlit sifferdetektion VG delen.....	14
5	Slutsatser	16
6	Teoretiska frågor.....	17
6.1	Uppdelning träning, validering och testdata set.....	17
6.2	Uppdelning av data set utan valideringsdata	17
6.3	Regressionsproblem inom maskininlärning	18
6.4	Hur kan du tolka RMSE och vad används det till	19
6.5	Beskrivning av klassifikationsproblem	19
6.6	Vad är K-means modellen, och var kan den tillämpas	21
6.7	Kategoriska data	21
6.8	Ordinal och Nominal variabler.....	22
6.9	Vad är Streamlit, och vad kan det användas till	22
7	Självutvärdering.....	23
7.1	Utmaningar du haft under arbetet samt hur du hanterat dem	23
7.2	Vilket betyg du anser att du skall ha och varför	23
	Källförteckning.....	25

1 Inledning

Maskininlärning, att skapa och använda intelligenta maskiner, har sitt ursprung runt 1950 när Alan Turing publicerade sin uppsats 'Computing Machinery and Intelligence', vilken innehöll det berömda Turing-testet för att avgöra om en maskin kan uppvisa beteende som inte går att särskiljas från en människas.

Maskininlärning har genomgått en stor transformation och kraftig marknadsexpansion, och används i många områden, såsom industrin, robotik, programutveckling, marknadsföring och försäljning och naturvetenskap och medicinska områden, men även underhållning och konst. En av det mest framstående exemplen är ChatGPT, inom avancerad naturligt språk-behandlingsmodeller, som generar svar utifrån användarens inmatning.

Vid utbildning och testning inom maskininlärning används ofta MNIST (Modified National Institute of Standards and Technology database), en stor databas, bestående av ett stort antal handskrivna siffror.

Denna rapport ingår som slutmoment, kunskapskontroll 2 i kursen Machine Learning, på utbildningen Data scientist, på EC utbildning Göteborg.

Syfte är att ge en översiktligt förståelse för maskininlärning en första inblick i processarbetet, att ta fram en modell för praktisk tillämplig.

- Lärande om teoretiska grunder, metoder och principerna, för hantering av ett multiklassificeringsproblem, att detektera siffror 0 – 9,
- Arbetet med modellval och optimeringar och justeringar för att lösa frågeställning.
- Genomföra hela processen designa, träna, utvärdera och använda maskininlärningsmodeller, från back-end till front-end till en streamlit applikation för sifferdetektion.

Frågeställningar

- Besvara de teoretiska frågorna som utgör första delen av kunskapskontrollen.
- Utveckla två maskininlärningsmodeller baserade på MNIST-datasetet och utvärdera deras prestanda.
- Skapa en Streamlit-applikation som integrerar en maskininlärningsmodell för att göra prediktion på ny osedd data, från användarens inmatning.

2 Teori

Problemställning sifferprediktion

MNIST-datamängden består av 70 000 bilder, handskrivna siffror och förklassificerad till 28 pixlar x 28 pixlar, siffrorna 0 - 9, där varje siffra utgör en klass, och består totalt av 10 klasser.

Multiklassificeringsproblem

Problemet är baserat på binär logik, True, False, och bygger därefter upp ett tillräckligt många binära teststeg för varje ingående klass som ska utvärderas. MNIST har 10 klasser, siffrorna 0-9 där grundfråga för algoritmen är Är talet X eller inte X (True, False)

2.1 Algoritmer vid klassificeringsproblem: One versus One, och One versus the Rest

En metod att lösa klassificeringsproblem är att dela upp problemet i flera binära klassificeringsproblem.

One vs one har metoden att dela upp samtliga klasser i samtliga kombinationer om två för jämförelse på binär nivå

0 vs 1, 0 vs 2 0 vs 3 1 vs 2 1 vs 3 2 vs 3 2 vs 4 2 vs 5

Under träning noterar modeller alltid vad som finns som skiljer sig mellan de två siffrorna i den binära klassen, när modellen får in en ny datapunkt, är uppgiften baserat på det som tidigare lärts, om den inkomna siffran är mer lika den första eller andra siffran i binärparet, och modellen tilldelar en röst den siffra som framstår som mest sannolik, och så arbetar processen vidare genom hela dataset för samtliga unika kombinationer av sifferpar.

Här finns också snarlika metoden, One versus the rest, modellen delar in i 10 olika dataset enligt nedan, och fungerar principiellt på samma sätt som OvO, med den skillnaden, indelning är gjord på en klass, jämfört med alla andra klasser.

Klass 0 siffran 0	mot siffrorna 1-9
Klass 1 siffran 1	mot siffrorna 2-9, samt 0
Klass 2 siffran 2	mot siffrorna 3-9 samt 0,1
Klass 3: siffran 3	mot siffrorna 4-9 samt 0, 1, 2
Klass 4 siffran 4	mot siffrorna 5-9 samt 0,1, 2, 3

2.2 Modeller inom multiklassificeringsproblem

Logistic regression är en välkänd beräkningsmetod inom statistik och ekonomi, främst utvecklad för att modellera samband och förutsäga sannolikheter, som kundbeteenden eller finansiella beslut. Den har också blivit populär inom maskininlärning och används ofta som grundmodell för klassificeringsproblem. Dess enkelhet, stabilitet och snabbhet gör den särskilt användbar i många tillämpningar, som har en rimlig kravnivå och utan högre grad av komplexitet.

Modellen löser klassificeringsproblem genom att uppskatta sannolikheter baserat på linjära samband mellan variabler. Den tillämpar detta återkommande för att hantera multiklassificeringsproblem, som exempelvis i MNIST-datasetet, där siffrorna 0–9 klassificeras. Logistic regression har behandlats och diskuterats i kursmaterial och undervisningsvideor och är en etablerad metod inom övervakad inlärning.

Nackdelar: En av de största begränsningarna är att modellen antar linjära samband, vilket blir problematiskt vid analys av komplexa dataset med icke-linjära strukturer, såsom bilder och ljuddata. Vid sådana tillfällen kan modellen ha svårt att konvergera till ett rimligt resultat och riskerar att ge låg prestanda och korrekta förutsägelser. Exempelvis kan dataset som CIFAR-10, med komplexa färgbilder av objekt, utmana modellens förmåga.

K-Nearest Neighbors, även denna modell har sin bakgrund från statistik och började utvecklas tidigt på 50-talet, och används ofta för klassificeringsproblem, är resultat är att bestämma vilken klass data tillhör. Modellens arbetssätt är baserat på antagande att ett objekt som ligger nära varandra tillhör samma klass. KNN skiljer sig från andra modeller, eftersom under träning, lagrar enbart datasetet, men utan någon modellering, en s k lazy learning.

Grundläggande är hyperparametern k , ofta ett lågt värde, som anger hur många grannar som ska inkludera vid beslutet.

Modellen är snabb och effektiv, och lätt att komma igång med har enbart ett fåtal in-parametrar, men kan fått problem snabbt vid stora dataset och när data är ojämt fördelat.

KNN användas i applikationer som tex rekommendation system, i spam mail filter, inom marknadsföring för kundsegmentering och för automatiserad transkriptioner, tal till skrift.

Beslutsträd, är en övervaknings träningsmodell som kan vändas både för regression och klassificeringsproblem. Vid prediktion, jobbar modellen med att stegvis ta sig från roten ut till ett löv, och för MNIST problemet motsvarar lövet en klass. Modellen är lätt att förstå och tolka, resultat kan bekräftas både med boolesk logik och statistiska metoder och lätt att visualisera träd.

Nackdelar, kan vara att om dataunderlaget är skevt fördelat att modell får en trädbias i någon riktning, så viktigt att förbehandla data setet med balansering, andra problem är modellen bygger en för komplex modell, i försök att tolka datat, vilket kan resultera i stället med overfitting och att modellen inte lyckas att generalisera väl.

En trädmodell används i flera olika tillämpningar, men är särskilt till fördel när det gäller diskreta beslut, en bedömning är antingen ja eller nej, samt det finns begränsad datamängd att tillgå.

Random forest, modellen är baserad i grunden baserat på en beslutsträdstruktur, som vidareutvecklats inom maskininlärning och räknas som en av de mest effektiva modellerna idag. Modellen har då godtyckligt antal träd, som bestämts av parametern $n_estimator$, som har default 100 träd, så att säga en ensemble beslutsträd modell med antal träd, bildar hel skog, för beslutsfattande.

Hist Gradient booster, också en ensemble modell av valfritt antal träd, som anges av parametern, max_iter , skillnaden mot RandomForest är att träden lära av tidigare misstag och justeras efter hand.

2.3 Optimering och hyperparametrar

Optimering handlar om att justera modellens hyperparametrar för att förbättra dess prestanda, här är första momentet att utvärdera vilka av modellen flertal hyperparameterar som ska utredas närmare.

- Grid Search och Random Search, de vanligaste, finns även mer avancerade metoder som använder Bayesisk optimering, tex Optuna.
- Fördelar: Hjälper modeller prestera bättre på både träning och testdata.
- Nackdelar: Kan vara tidskrävande för stora dataset och komplexa modeller.

2.4 Ensemble-teori:

- Ensemble-tekniker kombinerar flera modeller för att skapa en starkare och mer robust modell, exempel är modellerna Random Forest, AdaBoos, och kodmässigt sätts det samma med en övergripande voting classifier, som tar in beslut från de ingående modellerna, med antingen soft eller hard voting.
- Fördelar: Ökar träffsäkerheten och minskar risken för överanpassning.
- Nackdelar: Kan vara resurskrävande och svårare att tolka.

2.5 Utvärdering

För att utvärdera hur väl en maskininlärningsmodell utfört prediktioner, finns ett antal utvärdering mått som används vid modellering av klassificeringsproblem, modellen kan köras genom en classification report, ett bibliotek från scikit -learn som redovisar följande mått:

Accuracy: Noggrannhet, antal korrekta prediktioner / totalt antal gjorda prediktioner.

Precision: Hur ofta är en positiv prediktion korrekt och beräknas genom $TP / (TP + FP)$

Recall: Återkallning, Hur väl kan modellen hitta alla TP, och beräknas genom $TP / (TP + FN)$

F1: Balanserat samtliga de andra måtten, harmoniskt medelvärde, $2 * (precision * återkallelse) / (precision + återkallelse)$

Utöver den finns även metoder med crossvalidation scores, där modellen körs på validation data, för att se hur väl modellen lyckas generalisera.

Confusion matrix: En tabell i matrisform, där kolumnerna representera förutsagda klasserna, och raderna representerar de faktiska klasser, därifrån går det att utlösa detaljerat för vilka klasser 0-9 som modellen gör felaktiga prediktioner.

3 Metod

Huvuduppgifter för kunskapskontrollen² i machine Learning var att designa och testa minst två maskininlärningsmodeller baserat på MNIST, den stora sifferdatabasen. Problemet är sifferdetektionsproblem, siffror 0 – 9, ett multiklassifikationsproblem.

Datasetet för uppgiften laddades ner från OpenML, och görs med Python biblioteket Scikit-Learn, som har använt genomgående i arbetet.

3.1 EDA- MNIST

Därefter inspekteras och analyseras datas struktur och uppbyggnad, X (data) och Y (labels) har formen:

X shape: (70000, 784)

y shape: (70000,)

Det finns totalt 70 000 bilder, och varje bild har storlek 28 x 28) pixlar, som nedplattats till endimensionell vektor 748 kolumner.

Antal tomma bilder: 0

Index för tomma bilder: []

Här körs även kontroll, om databasen även innehåller några andra oväntade element

Finns NaN-värden i X_train?

Output: False

Finns oändliga värden i X_train?

Output: False

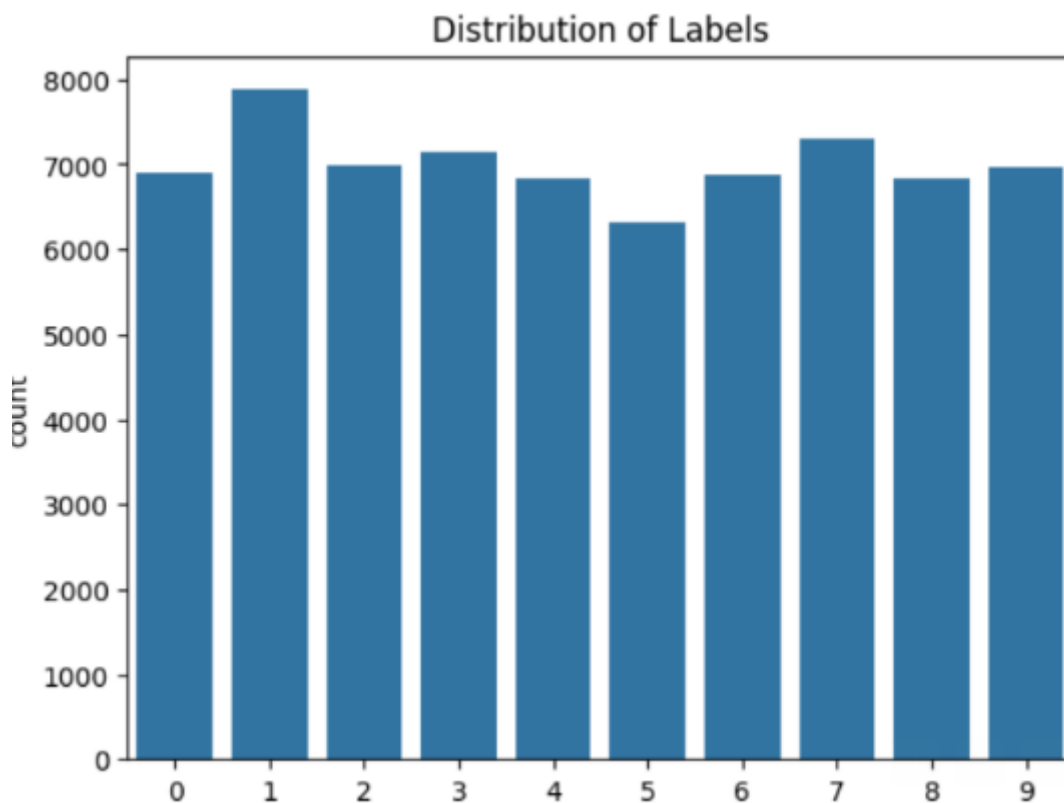
```
negative_values = X_train[X_train < 0]
```

```
print(negative_values)
```

Output []

Datasetet verkar stabilt och välorganiserat, så här långt, på grundläggande elementnivå.

Sammanställning, diagram från MNIST, fördelning per siffra



Fördelning av siffrorna, per antal stycken

```
0    6903
1    7877
2    6990
3    7141
4    6824
5    6313
6    6876
7    7293
8    6825
9    6958
Name: count, dtype: int64
```

Det framgår på detaljnivå, utskrift att materialet innehåller fler 1, 3 och 7, samt något längre antal av siffran 5.

Uppdelning av MNIST data set, träning, validering och testdata

Jag har valt att initialt träna modellerna på ett träningsdata på 25 000 bilder, se Python output.

Train data shape: (25000, 784), (25000,)

Validation data shape: (15005, 784), (15005,)

Test data shape: (29995, 784), (29995,)

3.2 Modellval

Jag har valt 6 maskininlärningsmodeller, av olika typer, med kapacitet att hantera klassificering problem. På grund av den långa tränings tiden, kontinuerliga krav av max iter, och den datakraften som krävdes, har jag exkluderat Support Vector Machine modeller från urvalet.

Logistic regression: En bra grundmodell för MNIST modellering, har behandlats och diskuteras i undervisningen, kursbok och videos, en snabb och stabil modell, med en enkel algoritm. Emellertid, med tydliga begränsningar inom områden såsom bildanalys.

Decision Tree: intressant att se hur en modell med beslutsträd, ett träd lyckas, algoritmen transverserande från rot upp till löven, se teoriavsnittet kap 2.

Random Forest: förbättrad modell av decision tree, intressant att ta med och se hur väl den lyckas på det tämligen grundläggande problemet. MNIS.

HistGradientBoosting Classifier: Ännu ett steg vidare på beslutsträd modellen, denna modell skiljer sig mot RandomForest bland annat genom att träden i denna modell, drar nytta av tidigare träd resultat och justera sitt data, därefter, medans RandomForest träd gör helt enskilda processer och röster därefter och enas om ett beslut.

KNeighborsClassifier : Modellen har jag valt som ett spännande experiment att utvärdera tanken att bedöma inkommande data baserat på dess X antal närmaste grannar , baserat på ett underlikande mönster i datat.

MultinomialNB

Default-inställning vid träning

De utvalda modellerna har tränats med grundinställningarna för hyperparametrar från Scikit-learn. Träningssprocessen har byggts upp med hjälp av en pipeline som inkluderar StandardScaler för dataskalning. Dessutom har hyperparametern `n_jobs = -1` använts för att tilldela varje modell maximalt tillgänglig datorkraft.

Följande kodexempel illustrerar träningsprocessen för modellen **Logistic Regression**:

```
1 log_reg = Pipeline([
2     ('scaler', StandardScaler()),
3     ('model', LogisticRegression(n_jobs=-1))
4 ])
5
6 log_reg_baseline = log_reg.fit(X_train, y_train)
7 score = log_reg_baseline.score(X_val, y_val)
8 joblib.dump(log_reg, 'log_reg_trained.pkl')
9 print('Modellen är tränad, och sparad.')
10 print("LogisticRegression:", score)
```

Modellen är tränad, och sparad.

3.3 Modell Utvärdering

Efter träning, utvärderades samtliga modeller med en score, här nedan visas resultat för Logistic Regression, 89,67 % på valideringsdata.

Modellen är tränad, och sparad.
LogisticRegression: 0.8967677440853049

3.4 Classification report

För samtliga modeller gjordes en utvärdering, i form av classification report som täcker några av de viktigaste utvärderingsmåten, precision, recall, samt f1 score. Nedan visas diagrammet för Logistic regression

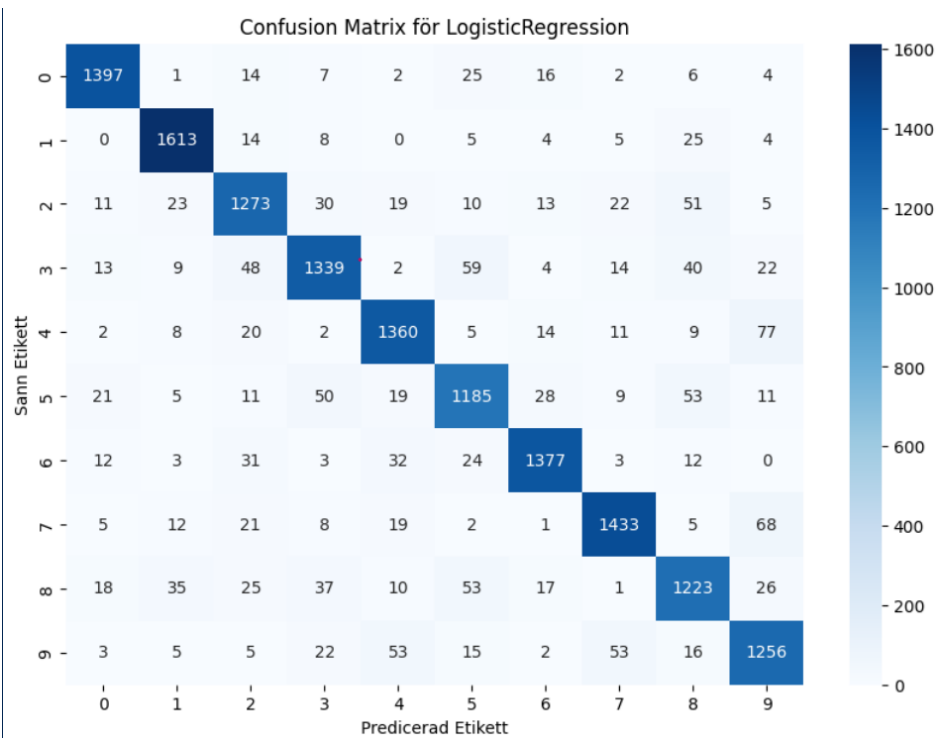
Classification Report för LogisticRegression					
	precision	recall	f1-score	support	
0	0.94	0.94	0.94	2966	
1	0.94	0.97	0.95	3380	
2	0.87	0.86	0.87	3004	
3	0.87	0.86	0.87	3026	
4	0.89	0.91	0.90	2844	
5	0.84	0.84	0.84	2699	
6	0.91	0.93	0.92	2975	
7	0.91	0.91	0.91	3185	
8	0.86	0.83	0.85	2887	
9	0.88	0.88	0.88	3029	
accuracy			0.89	29995	
macro avg	0.89	0.89	0.89	29995	
weighted avg	0.89	0.89	0.89	29995	

Cross validation score

```
1 # Ladda den sparade modellen
2 log_reg = joblib.load('log_reg_trained.pkl')
3
4 # Definiera scorer
5 scoring = {
6     'accuracy': 'accuracy',
7     'precision': make_scorer(precision_score, average='macro'),
8     'recall': make_scorer(recall_score, average='macro'),
9     'f1': make_scorer(f1_score, average='macro')
10 }
11
12 # Beräkna cross-validation scores
13 cv_results = cross_validate(log_reg, X_train, y_train, cv=5, scoring=scoring)
14
15 # Skapa en lista med resultaten
16 log_reg_results = []
17 for fold in range(5):
18     log_reg_results.append({
19         'Fold': fold,
20         'Accuracy': cv_results['test_accuracy'][fold],
21         'Precision': cv_results['test_precision'][fold],
22         'Recall': cv_results['test_recall'][fold],
23         'F1-score': cv_results['test_f1'][fold]
24     })
25
26 # Konvertera listan till en DataFrame
27 log_reg_results = pd.DataFrame(log_reg_results)
28
29 # Spara DataFrame med joblib
30 joblib.dump(log_reg_results, 'cross_validation_results.pkl')
31
32 # Visa DataFrame
33 log_reg_results
```

3.5 Confusion matrix

Som ett led på grundnivå, har respektive modells Confusion Matrix plottats och den illustrerar på detaljnivå hur väl modellen lyckas med respektive siffror och hur många ofta gissningarna är felaktiga. Infogat diagram, en confusion matrix för Logistic Regression



Det framgår att högsta siffran felaktiga prediktioner är 77 mellan siffrorna 4 och 9, totalt 77, och har sett de bekräftat också att det är vanligt att modellerna tar miste på just dessa två siffror.

Misstag mellan 4 och 9: 77
Misstag mellan 9 och 4: 53
Största prediktionsfelet: 0.00308

3.6 Prediktionstest baserat på 5 MNIST bilder

Samtliga modeller har körts för att gissa ett antal siffror från MNIS, verklig siffa true, samt predikterad siffra pred.

Prediktioner med LogisticRegression				
True: 9 Pred: 9	True: 6 Pred: 6	True: 5 Pred: 5	True: 5 Pred: 5	True: 8 Pred: 8
				

3.7 Optimering

De modeller som uppnådde en noggrannhet på över 90 %, på valideringsdata, valdes ut för vidare analys och optimering.

- RandomForestClassifier, HistGradientBoostingClassifier, KNeighborsClassifier

Urval väsentliga hyperparameter för KNeighborsClassifier:

- **K:** antal grannar, default värde är 5, min pythonkod testas körning k-värden i intervallet [3 – 9]
- **weights:** viktade beräkningsmetod: default värde, 'uniform' här testas intervallet ['uniform', None, 'distance']

Urval, väsentliga hyperparamerar för RandomForestClassifier

- **n_estimator:** default värde 100, antal träd i beräkningsmodellen, python kod testar antal [200,300, 400]
- **max_depth:** hur långt ner, djup ska modellen fortsätta beräkningen innan den anser sig har funnit bästa värdet, default värde None, python koden testar, range(25-28)

Urval, väsentliga hyperparamterar för HistGradientBoosteingClassifier

- **learning_rate :** standard värde: 0,1 multipliceringsfaktor löv, Pythonkoden testar intervallet [0,01 0,1 1]
- **max-iter:** n_classes av träd skapas per iteration, default värde, 100. Pythonkoden testar intervallet [202,203,205]

Normalt utförs optimering med gridsearch, men för en effektivare och fokuserad process, och inget krav på brute force och långa datakörningar, kördes istället en loop score variant. För varje hyperparameter definierades ett intervall av värden att testa, En loop itererade över dem, och beräknade score för respektive konfiguration, score för varje parameter-konfiguration sparade och högst parameter score valdes för omträning.

```
Träning av optimerad KNN-modell

k_values = ['distance', 'uniform', None]
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=3, weights = k, n_jobs = -1)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    print(f"K: {k}, Genomsnittlig noggrannhet: {scores.mean():.4f}")

✓ 2m 15.0s
```

Modellernas pipelines uppdateras med de funna och optimerade hyper-parametrarna, och tränas därefter om med träning data, och tränings + valideringsdata.

Därefter skapas en voter classifier, som använder soft voting, där samtliga tre modeller ingår, och den genomgår standardprocessen, träning, utvärdering, samt därefter sammanslagning data från träning och valideringssetet.

I det avslutande processteget körs samtliga fyra modeller på testdata setet och utvärderas därefter, resultat presenteras i kapitel 4.

Den här beskrivna arbetsprocessen, har genomförts för att säkerställa en grundlig och metodisk modellering av MNIST dataset för två maskininlärningsmodeller.

4 Resultat och Diskussion

I arbetet med att modellera MNIST användes totalt 6 modeller som utgångspunkt, varav de tre bästa på sammanlagt resultat hade över 90 %, enbart de 3 bästa gick vidare till optimering och körning på testdatat.

Namn	Score	Classification report Accuracy Precision, Recall, f1	Cross validation score Accu, prec, recall, F1 4-fold
Logistic Regression	0,90	0,89; 0,89; 0,89; 0,89	0,8938; 0,8927; 0,8925; 0,8925
Decision Tree	0,84		0,83; 0,83; 0,83; 0,83
Random Forest			
MultinomialNB	0,83	0,83; 0,83; 0,82; 0,83	0,83; 0,84; 0,83; 0,84
HistGradientBoosting			
KNeighborsClassifier			

Resultat efter optimering

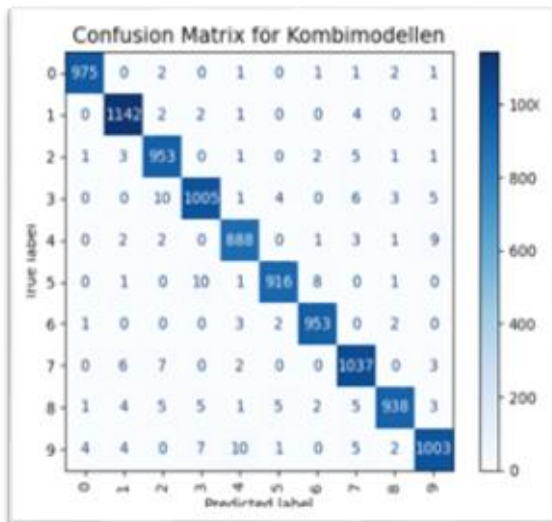
Namn	Score	Classification report	Cross validation score
RandomForest			
KNN			
HistGradientBoosting			

Efter optimering, skapas även en ensamble modell, via Voter classifier där samtliga modeller 3 ovan modeller ingår.

Resultat efter testkörning

Namn	Score	Classification report	Cross validation score
RandomForest			
KNN			
HistGradientBoosting			
Voter Classifier			

I arbetet på kursen Machine learning med den praktiska uppgiften, träna två modeller för MNIST, presterade maskininlärningsmodellerna Voter Classifier, och Hist-GradientBoostingClassifiing bäst med 98 % score vardera genomgående.

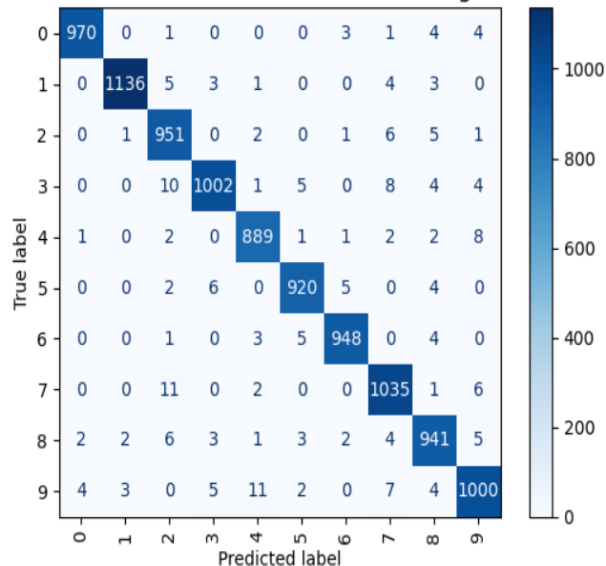


Klassificeringsrapport:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	983
1	0.98	0.99	0.99	1152
2	0.97	0.99	0.98	967
3	0.98	0.97	0.97	1034
4	0.98	0.98	0.98	906
5	0.99	0.98	0.98	937
6	0.99	0.99	0.99	961
7	0.97	0.98	0.98	1055
8	0.99	0.97	0.98	969
9	0.98	0.97	0.97	1036
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

cr för Kombimodellen är nu sparad som class_report.pkl

Confusion Matrix för HistGradientBoostingClassifier



Klassificeringsrapport:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	983
1	0.99	0.99	0.99	1152
2	0.96	0.98	0.97	967
3	0.98	0.97	0.98	1034
4	0.98	0.98	0.98	906
5	0.98	0.98	0.98	937
6	0.99	0.99	0.99	961
7	0.97	0.98	0.98	1055
8	0.97	0.97	0.97	969
9	0.97	0.97	0.97	1036
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

cr för HistGradientBooster är nu sparad som class2_report.pkl

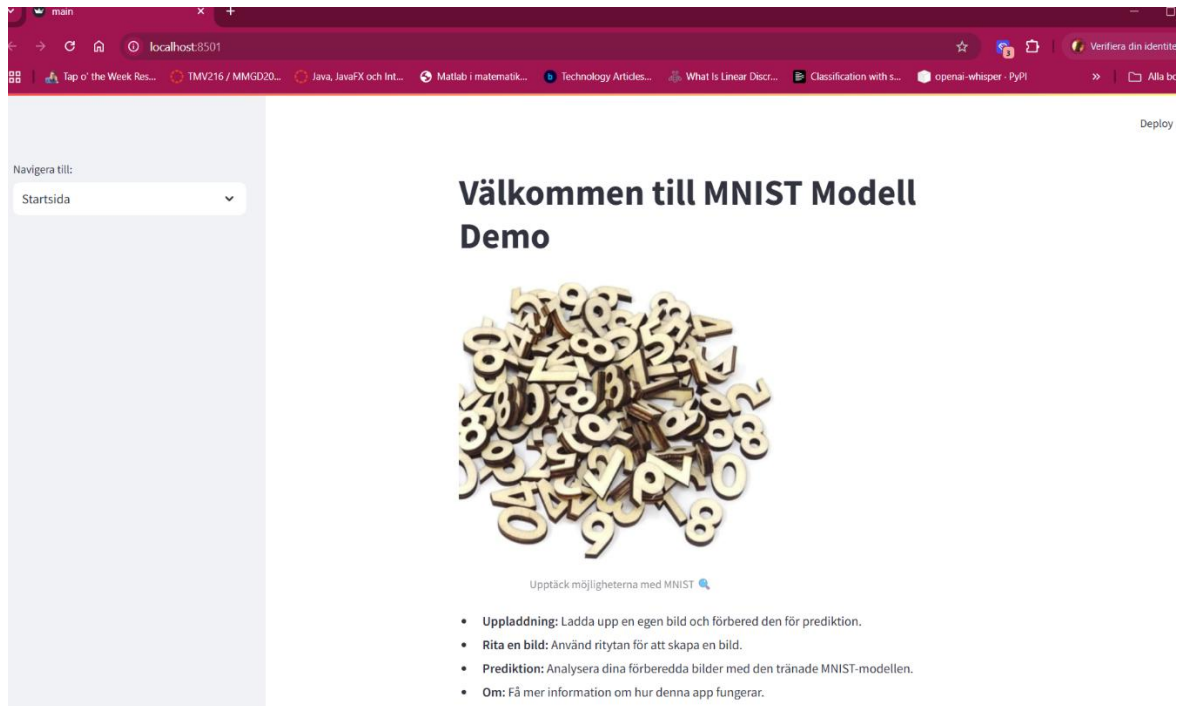
MNIST har ett antal fördelar, det lätt att komma i gång och börja bearbeta materialet, som är mycket väl strukturerat och stabilt, så fokus blir på själva modelleringen, och inte på dataprocessen, vilket kan vara en effektiv metod, både i forsknings och utbildningssammanhang.

Emellertid, kan man fråga om det är vettigt i längden att träna modeller att prestera toppresultat enbart i ett så begränsat området som MNIST, enkelt till sin natur, så det är lätt att få intrycket att modellen då enbart är bra att hantera MNIST och i sort sätt ingenting mer, och det bör gälla även inom forskningsvärlden, om man ska använda något som benchmark kanske det är dags att höja nivån nu 2025.

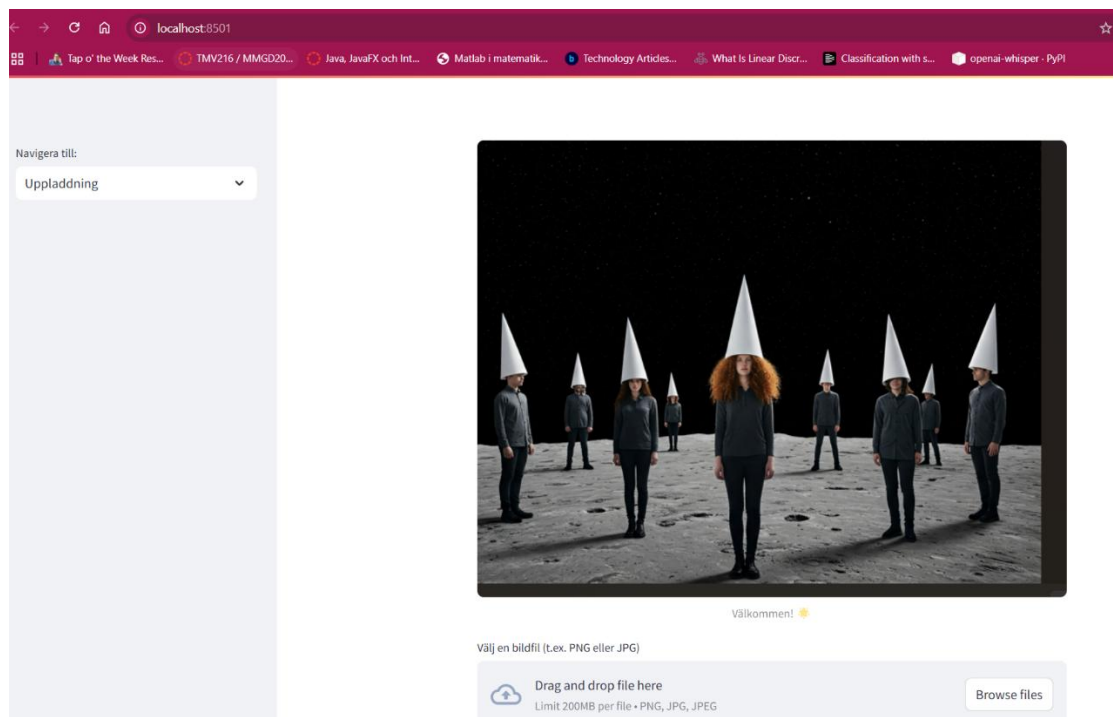
4.1 Streamlit sifferdetektion VG delen

I kontrolluppgiften frivilliga del, skapades en applikation för sifferdetektion i streamlit, där användaren kan lägga in nya siffror antingen via uppladdning eller rita in en siffra. För detektionen av siffror används modell som tränats i tidigare moment från MNIST.

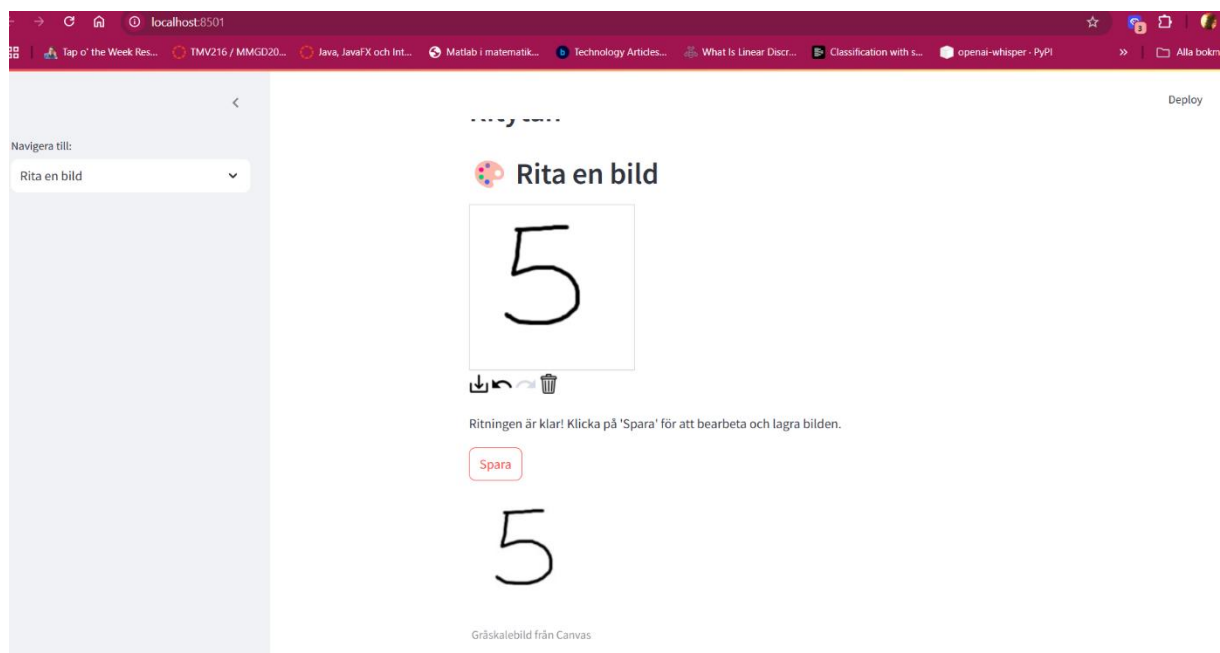
Screenshot appens första sida



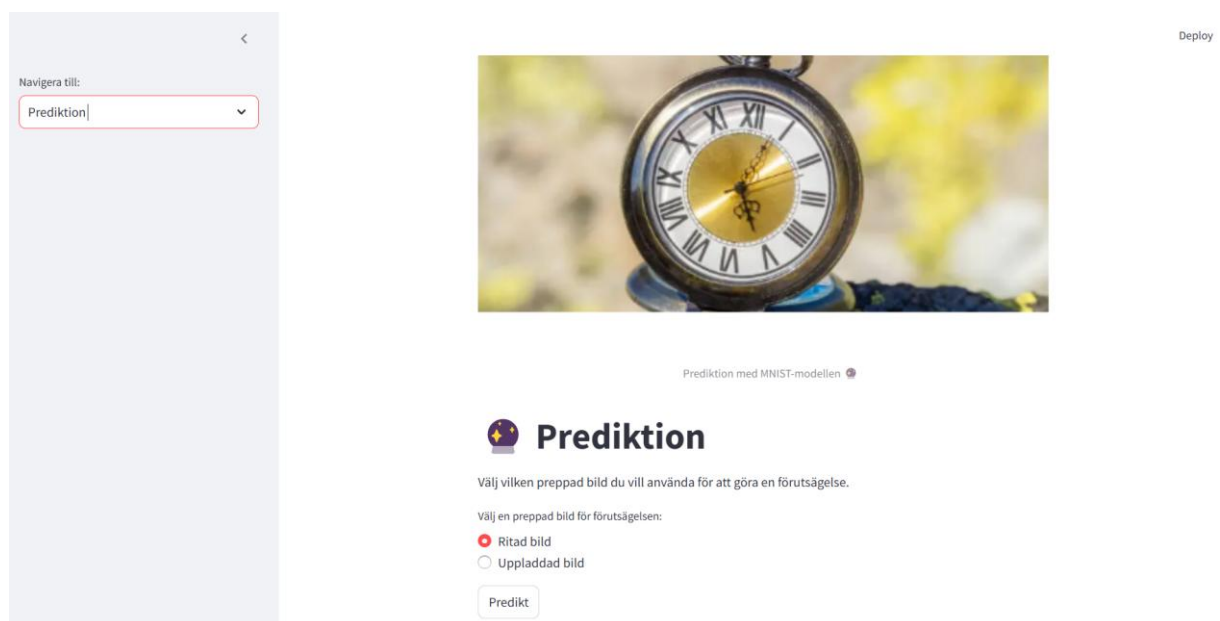
Screenshot appens andra sida, uppladdning sifferbild



Screenshot appens tredje sida, rita sifferbild



Screenshot appens fjärda sida, prediktion, antingen från uppladdad bild eller sparad ritad sifferbild.



Under arbetet med Stramlit applikationen, där en framtagen inlärningsmodell, ska användas på helt osedda data, framgår problemet på många fronten, att MNIST databasen, inte speglar de komplexiteten som finns i verkliga sifferdetektionsproblem, såsom olika handstilar, dåligt ljus eller brus i bilder.

Lösningen är att träna om modellen, genom att blanda upp det väldigt homogena datasetet MNIST med egna handtränade bilder med olika pennstorlekar och tjocklek och vinklar, samt data augmentation, samt även att fundera i termer av bildbehandling så de nya bilderna får en så MNIST liknade format som möjligt.

5 Slutsatser

I arbetet har datasetet MNIST delats upp enligt teori fråga 6:1 i träning, validering och test. Denna trestegsprocess har följts då den är ett etablerat och lämpligt angreppssätt för klassificeringsproblem, istället för att enbart använda träning och test, vad som beskrivs teoriavsnitt 6.2

Arbetet har inte fokuserat på regressionsproblem eller använt modeller av typen linjär regression, då den praktiska frågeställningen inriktade på klassificeringsproblem, sifferdetektion.

Utvärderingsmåttet RMSE, är inte lämpligt, eftersom det primärt används för regressionsproblem, för uppgiften med sifferdetektion, Istället har mått som noggrannhet (accuracy), precision och F1-score använts för att bedöma modellens prestanda, relevanta och effektiva för typen av problem.

Sifferdetektion är ett multiklassificeringsproblem som involverar 10 klasser, siffrorna 0-9. För att lösa problem har modeller som Random Forest, KNN, Histogram-Based Gradient Boosting, Decision Tree och Logistic Regression använts. Dessa modeller är valda eftersom de är särskilt väl lämpade för att hantera klassificeringsproblem och har kapacitet att effektivt separera flera klasser i datan, För att utvärdera samtliga modeller har Confusion Matrix använts, vilket ger en detaljerad insikt i deras prestanda genom att belysa både korrekta och felaktiga klassificeringar.

En möjlig alternativ metod, hade kunnat vara att använda K-means-klustering för att identifiera naturliga grupperingar i siffrorna, teorifråga 6.6 och styra upp $k = 10$, men i detta arbete har problemet betraktats som ett klassificeringsproblem primärt.

Samtliga siffror i datasetet är av nominal typ, då det inte finns någon inbördes ordning mellan dem. Eftersom siffrorna representerar separata klasser, har det inte varit nödvändigt att använda någon typ av dummyvariabelkodning.

I steg två har två modeller på MNIST tagits fram, de är baserade på HistGradientBoosting och Voter classifier, och slutligen en streamlit applikation byggts där en maskininlärningsmodell har inkluderats för att göra sifferprediktioner på osedd datainformation, genom att användarens inmatning.

6 Teoretiska frågor

6.1 Uppdelning träning, validering och testdata set

Kalle delar upp data i ”Träning”, ”Validering” och ”Test”, vad används respektive för?

Träningsdata: För att utveckla ett Machine Learning -system, används träningsdata. Modellen analyserar datasetet för att förstå dess innehåll och struktur. Genom processen beräknas inlärningsbara parametrar utifrån given information, vilket gör det möjligt för modellen identifiera mönster och lär sig från datan.

Valideringsdata: För att bedöma modellens prestanda efter träning, Det ger en indikation på hur väl modellen har anpassat sig till uppgiften och hjälper till att identifiera områden där förbättringar kan behövas, och valideringsdata används också för att minimera både under och överanpassning.

Testdata Under testfasen använd nya, tidigare osedda data för att utvärdera modellens förmåga att hantera praktiska tillämpningar, vilket säkerställer att modellen uppfyller förväntade prestandakrav innan den implementeras och sätts i bruk eller släpps ut på marknaden.

6.2 Uppdelning av data set utan valideringsdata

Julia delar upp sin data, på träningsdata tränar hon tre modeller; ”Linjär Regression”, ”Lasso regression” och en ”Random Forest modell”. Hur skall hon välja vilken av de tre modellerna hon skall fortsätta använda när hon inte skapat ett explicit ”validerings-dataset”?

Problemanalys:

Valet beror på frågan som ska utredas, gäller ett linjärt samband, med syftet att förutspå y givet flera in-parametrar

Formeln för linjär regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Modellerna som analyserar det är Linear Regression” och Lasso Regression, efter en gridsearchcv, väljs modellen Lasso regression för utvärdering.

Gäller det istället ett icke linjärt samband, eller någon form av klassificerings-problem, är det lämpligt, att efter träning och optimering fortsätta använda med Random Forest, för utvärdering

Utvärdera på träningsdata: Det är möjligt att efter träning, köra de tre modellerna på classification report, samt cross-validerings scores för att se vilken som ger bäst prestanda baserat på träningsdata.

Utifrån resultaten väljs den modell som uppvisar sammantaget det bästa resultatet av *classification report* och *cross-validation scores* för vidare utvärdering.

6.3 Regressionsproblem inom maskininlärning

Regressions problem, en statistikmodell som tillhör kategorin övervakad inlärning, och har målet att med hjälp av en eller flera invariabelt som föreslå ett värde som numeriskt output, baserat ett samband mellan variablerna,

Beroende variabel (Target) ofta betecknas Y: det som ska prediktera

Oberoende variabler (Features (ofta betecknas X: de variabler som påverkar Y värdet,

Regression förekommer av en rad olika typer:

- Enkel linjär regression
- Multipel linjär regression
- Polynomial regression

Exempel på modeller inom regressionsproblem

LinearRegression	Support Vector Regression:	Random Forest Regression
Lasso Regression		Gradient Boosting Regression
Ridge Regression	Decision Tree Regression	

Tillämpningsområden

Olika typer av försäljningspriser (tex hus)	Kund churning, försäljning och marknadsföring
Kundbeteende, Marknadsföring	Inkomst, vikt, temperatur, vetenskapliga området, väder, och vindförhållanden, och olika typer av meteorologiska prediktioner
Befolkningsmängd samhällsvetenskapliga prediktioner.	Vikt, risk för sjukdomar, olika typer av medicinska studier.
Nationalekonomi, macro analyser prognoser.	
Ekonomi och finans, aktiemarknads prognoser, riskhantering kundlivscykel	

6.4 Hur kan du tolka RMSE och vad används det till

$$RMSE = \sqrt{\sum_i (y_i - \hat{y}_i)^2}$$

RMSE, root mean square error, centralt inom statistik, och är standardavvikelsen av hur långt från den tänkta regressionslinjen samtliga data punkt ligger, dvs totala prediktionsfelet. Avståndet, räknas för varje datapunkt, kvadreras och sen divideras med antal data punkt, medelfelet, och därefter tas kvadratroten av resultat.

RMSE används huvudsakligen som utvärderingsmått för linjära regression modeller, och lägre RSME värde ju bättre är modellen på att prediktera framtida utfall, och är särskilt viktigt mått vid en applikation där det är särskilt viktigt att undvika stora prediktionsfel på modellen, används även för att jämföra bland ett antal modeller, vem som kan anses vara mest tillförlitlig att matcha datasetet.

6.5 Beskrivning av klassifikationsproblem

Binär klassifikation

Grundläggande principen för enkel klassificering är genom att dela upp problemet i två klasser, ett binärt urval 1 eller 0, där linsparametern kan anta värdet antingen True eller False, vanligt exempel här är kund churning och spam-mail, resultat har två klasser.

1: Har, eller kommer kunden att lämna bolaget: som kan anta antingen True eller False

2: Är mailet spam: kan anta antingen True eller False

Multiklass klassifikationer: Det går att använda binära klassiferares för att köra multiklass på problem och det görs genom algoritmerna One versus the rest (OvR) och One versus One (OvO).

Modeller som löser klassifikationsproblem

DecisionTreeClassifier	KNeighborsClassifier	RandomForestClassifier
HistGradientBoosterClassifier		

Tillämpningsområden, klassifikation

MNIST, sifferprediktion, spammail Ekonomi, fraud detektion Ekonomiska prognoser, aktiemarknad	Vetenskap: Bildigenkänning, experimentell analys, DNA sekvenser för biologisk forskning, patient fatalitetsrate, Detektion kön/bilder
Marknadsföring, churn, segmentering kampanjutvärdering, och simulering försäljning och prognos data, simulering	Medicin: prognos, diagnos, övervakning, patientövervakning, framtagning av nya läkemedel och utvärdering

Beskrivning förvirringsmatris

Confusion matrix, är ett viktigt verktyg för att utvärdera klassificeringsproblem, och har två dimensioner,

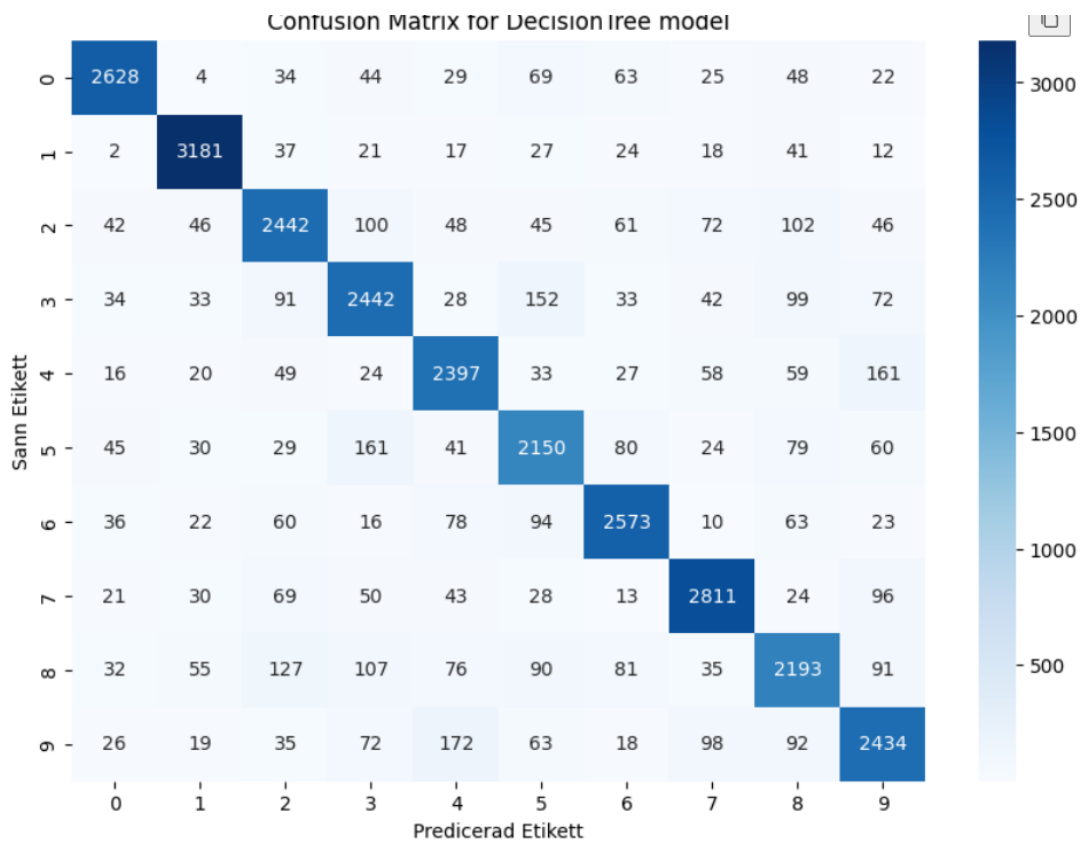
Faktiskt utfall, till vänster märkt Sann Etikett

Predikterad Etikett, längst ner

Här ser man på blå diagonalen var utfall och predikterat utfall samma och modeller har lyckats att korrekt att prediktera en 0 då det också var sant värde var noll, det samma gäller för resterande diagonalvärden 1–9.

Man kan gå in för samtliga värden och se hur många gånger av datasets körning X, har totalt modellen gissat på ett visst värde.

Sök upp siffran 172 i matrisen, vilket är det antalet gånger en sann siffra, 9 har modellen predikterat felaktigt som en 4, värden avläses på respektive X och Y skalan.



Utvärderingsmått från modeller, Precision recall och F1 är samtliga baserat på mått och värde från confusion Matrix.

6.6 Vad är K-means modellen, och var kan den tillämpas

K-means, har sitt ursprung från signalbehandling, och är en oövervakad inlärningsmetod, som bygger på metoden om klustring. Modellen beskrivs som enkel och användbar inom de flesta områden där data analyseras.

Metoden som modellen arbetar efter är dela in, kategorisera stycken observation i K stycken kluster, och det görs genom beräkning av ett avstånd från klustrets centrum, och syftet är att minimera summan av avståndet, och hitta en optimal indelning av data i olika grupper.

Tillämpningar: Företags och marknadsföring, kundsegmentering, bildsegmentering, anomalidetektering och genanalys. Enkel och effektiv för många datatyper, biologiska dataanalys, logistik och supply chain optimeringar.

6.7 Kategoriska data

Förklara följande gärna med kodexempel

Ordinal encoding: Variabler som har klass, behöver i många fall göras om till ett numeriskt värde, så varje variabel tilldelas en siffra, och i många fall är det tillräckligt för att algoritmen ska kunna hantera och rangordna dessa, och det finns naturligt inbördes rangordning, variabeln är av ordinal typ.

Utdrag kodexempel: Klassen placering

'först' = 1, 'andra' = 2, 'tredje' = 3 eller trafikljus

Klassen uppmärksamhet i trafiken

'Rött' = 1, 'orange' = 2, 'grönt' = 3

One-hot encoding: När det istället gäller datavariabler av nominal typ, där det inte existerar något inbördes förhållande eller rangordning, behövs representationen göras på ett annat sätt, tex här på matrisform, en binär variabel för varje kategori.

Antal variabler antal kolumn i matrisen

Variabel namn	Blå	Grön	Röd
Röd	0	0	1
Grön	0	1	0
Blå	1	0	0

Dummy variable encoding: Denna metod är tillämplig för modell linjär regression, och representerar X antal variabler med X-1 binära variabler.

Exempel från ovan, tre variabler har ju 4 binära alternativ och det räcker väl till för att representera dem, utan redundans.

Kod exempel

Röd 00, blå 01, grön 10, oavsett fördelning, med denna metod används alla skapade variabler utom en.

6.8 Ordinal och Nominal variabler

Göran påstår att datan antingen är ”ordinal” eller ”nominal”. Julia säger att detta måste tolkas. Hon ger ett exempel med att färger såsom {röd, grön, blå} generellt sett inte har någon inbördes ordning (nominal) men om du har en röd skjorta så är du vackrast på festen (ordinal) – vem har rätt?

Data brukar delas upp i två huvudgrupper: numeriska och kategoriska, där kategoriska data i sin tur består av undergrupperna ordinal och nominala variabler.

När kategoriska variabler har en inbördes naturlig rangordning, och klassificeras de som ordinala.

Julia argumenterar: Det finns ingen naturlig inbördes ordning mellan färgerna röd, grön och blå, men om man vill vara snyggaste klädd bör man välja röd, och då på något sätt skulle den slutsatsen göra röd till ordinal.

Det stämmer förstås inte, det finns ingen naturlig (dvs objektiv rangordning) mellan färger, hon ger heller ingen tydlig anledning varför röd skulle byta klassificeringstyp.

Göran har rätt, det finns ingen inbördes rangordning av data, där variablerna betecknas av olika färger, även om jag personligen också tycker att röd är snyggaste färgen.

6.9 Vad är Streamlit, och vad kan det användas till

Streamlit, ett open-source Python bibliotek, särskilt anpassat för data scientist och AI och maskininlärningsingenjörer, och används för att bygga och dela interaktiva applikationer och webbsites, på ett enkelt och smidigt sätt, med fokus på användarvänlighet och snabb prototyputveckling.

7 Självutvärdering

7.1 Utmaningar du haft under arbetet samt hur du hanterat dem

- Data processkraft
- Långa python körningar, kodoptimering, dead kernels, Anaconda krasch, ominstallation
- Colap
- Gridsearch optimeringar

7.2 Vilket betyg du anser att du skall ha och varför

Rapporten uppfyller alla kursmål och visar på en gedigen förståelse för maskininlärning, både teoretiskt och praktiskt. Arbetet inkluderar implementering och utvärdering av flera modeller, som redovisas i *Kapitel 2*, *Kapitel 3* och *Kapitel 4*, med resultat som uppnår hög noggrannhet och välmotiverade modellval. Dessutom visar diskussionen i *Kapitel 5* och *Kapitel 6* på en förmåga att kritiskt analysera och reflektera över modellernas prestanda och begränsningar.

Den framtagna Streamlit-applikationen visar på praktisk tillämpning av teori och modeller, medan självutvärderingen i *Kapitel 7.1* visar att utmaningar hanterades med framgång, vilket demonstrerar en hög grad av självständighet.

Sammanfattning och betygsförslag: Med tanke på rapportens kvalitet, omfattning och djup anser jag att betyget **VG (Väl Godkänt)** kan vara motiverat.

Appendix A

1. Python koden för EDA och två modeller från MNIST
Länk

<https://github.com/ykpersson/ML>

Källförteckning

[Ordinal and One-Hot Encodings for Categorical Data - MachineLearningMastery.com](https://machinelearningmastery.com/ordinal-one-hot-encodings-categorical-data/)